

Lab Assignment 09 (Authorization & Constraint Mechanics)

Department: CSE

Subject: DIS Lab

Code: 22CPP217

Objective

To implement and analyze **database authorization mechanisms using GRANT and REVOKE**, understand **privilege delegation**, and experimentally test **constraint behavior such as UNIQUE, DEFAULT, NOT NULL, and foreign key actions like ON DELETE CASCADE and ON DELETE SET NULL**.

Pre-Lab Setup

Run this block to initialize the environment.

```
-- Reset Environment
SET SQL_SAFE_UPDATES = 0;
DROP TABLE IF EXISTS Enrollments, Courses, Students;

-- Students Table
CREATE TABLE Students (
    Student_ID INT PRIMARY KEY,
    Student_Name VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    City VARCHAR(50) DEFAULT 'Jaipur'
);

-- Courses Table
CREATE TABLE Courses (
    Course_ID INT PRIMARY KEY,
    Course_Name VARCHAR(50)
);

-- Enrollments Table
CREATE TABLE Enrollments (
    Enroll_ID INT PRIMARY KEY,
    Student_ID INT,
```

```

Course_ID INT,

CONSTRAINT fk_student FOREIGN KEY (Student_ID)
REFERENCES Students(Student_ID)
ON DELETE CASCADE,
CONSTRAINT fk_course FOREIGN KEY (Course_ID)
REFERENCES Courses(Course_ID)
ON DELETE SET NULL
);

-- Insert Sample Data
INSERT INTO Students (Student_ID, Student_Name, Email, City) VALUES
(101, 'Aman', 'aman@email.com', 'Delhi'),
(102, 'Riya', 'riya@email.com', 'Mumbai'),
(103, 'Kabir', 'kabir@email.com', 'Jaipur');

INSERT INTO Courses VALUES
(1, 'DBMS'),
(2, 'Operating Systems'),
(3, 'Computer Networks');

INSERT INTO Enrollments VALUES
(1, 101, 1),
(2, 102, 2),
(3, 103, 3);

```

Q1: Testing Basic Authorization

The administrator wants to control **read-only access** to student data.

- Write a **GRANT statement** allowing **userA** to perform **SELECT on the Students table**.
 - As **userA**, run a **SELECT** query to verify that the user can view the records.
 - Attempt to execute a **DELETE** operation on the **Students** table as **userA**.
 - Record the **exact MySQL error message** that prevents the operation.
 - Write a **REVOKE statement** to remove the **SELECT** privilege from **userA** and verify that the user can no longer read the table.
-

Q2: Privilege Delegation

The administrator wants to test **delegated privileges**.

- Grant **SELECT privilege on Students to userB WITH GRANT OPTION**.
 - As **userB**, grant SELECT privilege on Students to another user **userC**.
 - As **userC**, run a SELECT query on the Students table to verify access.
 - Revoke the SELECT privilege from **userB** and observe whether **userC** still has access. Document the result and explain how **privilege propagation behaves after revocation**.
-

Q3: Authorization using Views

Sensitive attributes such as **Email addresses should not be accessible to normal users**.

- Create a **view named Student_Public** containing only:
 - Student_ID
 - Student_Name
 - City
 - Grant **SELECT privilege on the view to userD**.
 - As **userD**, run a SELECT query on the view and verify the returned data.
 - Attempt to access the Email column directly from the Students table as **userD** and document the DBMS response.
-

Q4: Mechanics of Constraints

This exercise demonstrates how constraints enforce data integrity and referential behavior.

- Attempt to insert a student record without specifying Student_Name and record the error caused by the NOT NULL constraint.
- Insert a student record without specifying City and verify that the DEFAULT value 'Jaipur' is automatically assigned.

- Attempt to insert another student with an existing Email value and record the UNIQUE constraint error message.

- Recreate the Enrollments table with the following foreign key actions:

```
CREATE TABLE Enrollments_New (  
    Enroll_ID INT PRIMARY KEY,  
    Student_ID INT,  
    Course_ID INT,  
    FOREIGN KEY (Student_ID) REFERENCES Students(Student_ID)  
        ON DELETE CASCADE,  
    FOREIGN KEY (Course_ID) REFERENCES Courses(Course_ID)  
        ON DELETE SET NULL  
);
```

- Insert sample data into Enrollments_New similar to the original table.
 - Delete a student record from the Students table and observe the effect on Enrollments_New.
 - Record how ON DELETE CASCADE affects the related records.
 - Delete a course record from the Courses table and observe the effect on Enrollments_New.
 - Record how ON DELETE SET NULL modifies the related records.
 - Write SELECT queries before and after deletion to clearly show the changes in the Enrollments_New table.
-

SQL Syntax Hints

```
-- Granting Privileges  
GRANT SELECT ON table_name TO user_name;  
  
-- Grant with Delegation  
GRANT SELECT ON table_name TO user_name WITH GRANT OPTION;  
  
-- Revoking Privileges  
REVOKE SELECT ON table_name FROM user_name;  
  
-- Creating a View  
CREATE VIEW view_name AS  
SELECT column1, column2 FROM table_name;  
  
-- NOT NULL Constraint Example  
Student_Name VARCHAR(50) NOT NULL;
```

```

-- Default Constraint Example
City VARCHAR(50) DEFAULT 'Jaipur';

-- UNIQUE Constraint Example
Email VARCHAR(100) UNIQUE;

-- Creating Table with Foreign Key Actions
CREATE TABLE Enrollments_New (
    Enroll_ID INT PRIMARY KEY,
    Student_ID INT,
    Course_ID INT,
    FOREIGN KEY (Student_ID) REFERENCES Students(Student_ID)
        ON DELETE CASCADE,
    FOREIGN KEY (Course_ID) REFERENCES Courses(Course_ID)
        ON DELETE SET NULL
);

-- Insert Data into New Table
INSERT INTO Enrollments_New VALUES (1,101,1);

-- Observe Data
SELECT * FROM Enrollments_New;

-- Delete Parent Record (CASCADE effect)
DELETE FROM Students WHERE Student_ID = 101;

-- Delete Parent Record (SET NULL effect)
DELETE FROM Courses WHERE Course_ID = 1;

```